《计算机图形学及人机交互》

实验报告

学　号：2023112621

姓　名：程泽凯

年　级：2023 级

专　业：计算机科学与技术

院　系：计算机与人工智能学院

2025 2026 第 1 学期

目录

1 实验目的、任务及要求 3
1.1 实验目的 . . . 3
1.2 实验任务与要求 . . . 3

2 实验环境 4
2.1 硬件环境 . . . 4
2.2 软件环境 . . . 4

3 主要功能模块结构及说明 5
3.1 系统总体架构 . . . 5
3.2 核心模块详细说明 . . . 5
3.2.1 几何体生成模块 . . . 5
3.2.2 图形变换模块 . . . 6
3.2.3 光照渲染模块 . . . 6
3.2.4 材质纹理模块 . . . 7
3.2.5 用户交互模块 . . . 8

4 实验算法及程序流程 9
4.1 三维几何体生成算法 . . . 9
4.1.1 球体生成算法 . . . 9
4.1.2 圆柱体生成算法 . . . 10
4.1.3 立方体生成算法 . . . 11
4.1.4 四面体生成算法 . . . 12
4.2 三维图形变换算法 . . . 12
4.2.1 世界矩阵计算 . . . 12
4.3 摄像机视图矩阵算法 . . . 13
4.4 射线拾取算法 . . . 14

5 程序实现 16
5.1 算法设计思路 . . . 17
5.1.1 Direct3D 11 渲染管线设计 . . . 17
5.1.2 着色器设计思路 . . . 17
5.1.3 材质系统设计 . . . 17
5.2 核心代码实现 . . . 18
5.2.1 顶点和常量缓冲区结构 . . . 18

5.2.2 世界矩阵计算 18
5.2.3 球体生成实现 19
5.2.4 圆柱体生成实现 20
5.2.5 四面体生成实现 22
5.2.6 顶点着色器实现 23
5.2.7 像素着色器实现 24
5.2.8 摄像机视图矩阵实现 26
5.2.9 射线拾取实现 26
5.2.10 材质预设实现 27
5.2.11 纹理加载实现 28
5.2.12 渲染循环实现 30

6 实验总结 33

1 实验目的、任务及要求

1.1 实验目的

通过本次实验，学习并实现计算机图形学中的三维图形生成与变换技术，掌握三维图形的建模、渲染和交互操作方法。

1. 掌握三维基本几何体的生成方法：球体、圆柱体、立方体、四面体、平面等
2. 理解三维图形的顶点、法线、纹理坐标等属性的计算方法
3. 掌握三维图形的几何变换：平移、旋转、缩放
4. 学习 Direct3D 11 图形渲染管线的基本原理
5. 掌握 Phong 光照模型的实现方法
6. 学习材质系统和纹理映射技术
7. 实现三维场景的交互式操作：图形选择、拖动、摄像机控制

1.2 实验任务与要求

1. 三维几何体生成：实现球体、圆柱体、立方体、四面体、平面等基本几何体的生成
2. 图形变换：实现三维图形的平移、旋转、缩放变换
3. 光照渲染：实现 Phong 光照模型，包括环境光、漫反射、镜面反射
4. 材质系统：实现多种预设材质（金属、塑料、玻璃、黄金等）
5. 纹理映射：支持从文件加载纹理并映射到几何体表面
6. 交互操作：实现鼠标拾取、拖动移动、摄像机旋转缩放等交互功能
7. 坐标轴显示：绘制三维坐标轴辅助定位

2 实验环境

2.1 硬件环境

1. 计算机：机械革命旷世 16pro
2. CPU：12th Gen Intel® i9-12900H
3. RAM：32GB
4. 显卡：NVIDIA GeForce RTX 4070 Laptop GPU

2.2 软件环境

1. 操作系统：Windows 11 专业版
2. 开发工具：Visual Studio 2022
3. 编程语言：C++17
4. 图形 API：Direct3D 11 (Windows SDK)
5. 着色器语言：HLSL (High Level Shading Language)
6. 图像加载：Windows Imaging Component (WIC)
7. 项目类型：Win32 桌面应用程序

3 主要功能模块结构及说明

3.1 系统总体架构

本系统采用模块化设计，基于 Direct3D 11 图形 API 实现三维图形的生成、渲染和交互。系统主要包括几何体生成、图形变换、光照渲染、材质纹理和用户交互五大功能模块。

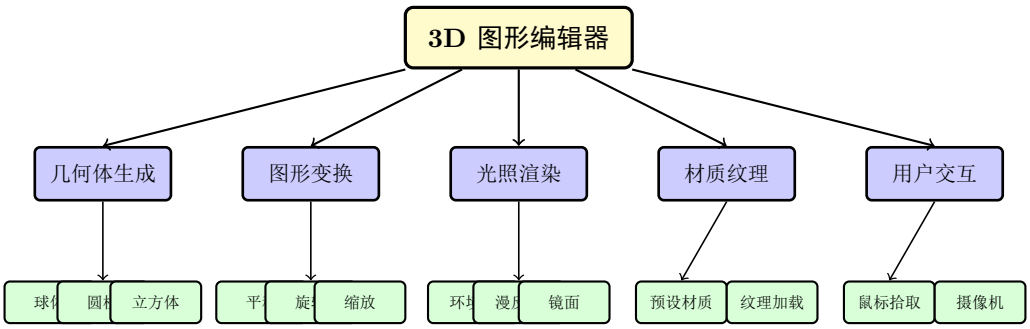


图 1: 系统模块结构图

3.2 核心模块详细说明

3.2.1 几何体生成模块

模块功能：
本模块实现多种三维基本几何体的生成，包括顶点坐标、法线向量、纹理坐标的计算。
支持的几何体类型：

- 球体 **(Sphere)**：使用经纬度参数化方法生成，支持自定义细分级别
- 圆柱体 **(Cylinder)**：包含顶面、底面和侧面，支持自定义半径和高度
- 立方体 **(Cube)**：六个面各自独立法线，支持纹理映射
- 四面体 **(Tetrahedron)**：正四面体，每个面独立法线
- 平面 **(Plane)**：双面渲染的矩形平面

顶点数据结构：

- Position：三维坐标 (x, y, z)
- Normal：法线向量，用于光照计算

- Color：顶点颜色 (RGBA)
- TexCoord：纹理坐标 (u, v)

3.2.2 图形变换模块

模块功能：
本模块实现三维图形的基本几何变换，所有变换都基于 4×4 齐次变换矩阵实现。
平移变换：

- 将图形沿指定方向移动指定距离
- 变换矩阵：$T(t_x, t_y, t_z) = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$

旋转变换：

- 支持绑定 X、Y、Z 三个轴的独立旋转
- 绕 X 轴旋转：$R_x(\theta) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$

缩放变换：

- 支持非均匀缩放（各轴独立缩放因子）
- 变换矩阵：$S(s_x, s_y, s_z) = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$

复合变换：世界矩阵 = 缩放 × 旋转 X × 旋转 Y × 旋转 Z × 平移

3.2.3 光照渲染模块

模块功能：
本模块实现 Phong 光照模型，在像素着色器中计算每个像素的最终颜色。
Phong 光照模型组成：

- 环境光 **(Ambient)**：模拟间接光照，$I_a = k_a \cdot L_a$
- 漫反射 **(Diffuse)**：与光线入射角相关，$I_d = k_d \cdot L_d \cdot \max(0, \vec{N} \cdot \vec{L})$
- 镜面反射 **(Specular)**：产生高光效果，$I_s = k_s \cdot L_s \cdot \max(0, \vec{V} \cdot \vec{R})^n$
- 自发光 **(Emissive)**：物体自身发出的光，$I_e = k_e$

最终颜色计算：

$$I = I_a + I_d + I_s + I_e = (k_a + k_d \cdot (\vec{N} \cdot \vec{L}) + k_s \cdot (\vec{V} \cdot \vec{R})^n) \cdot C_{base} + k_e$$

光照参数：

- 光源方向：可调节的平行光方向向量
- 光源颜色：RGB 颜色值，支持超过 1.0 的 HDR 值

3.2.4 材质纹理模块

模块功能：
本模块实现材质属性管理和纹理映射功能。
材质属性：

- 环境光系数 (Ambient)：控制环境光反射强度
- 漫反射系数 (Diffuse)：控制漫反射颜色
- 镜面反射系数 (Specular)：控制高光颜色和强度
- 自发光系数 (Emissive)：控制自发光颜色
- 光泽度 (Shininess)：控制高光范围大小
- 透明度 (Transparency)：控制物体透明程度
- 反射率 (Reflectivity)：预留的反射属性

预设材质样式：金属、塑料、玻璃、橡胶、黄金、白银、紫铜、铬、翡翠、绿宝石、珍珠等 11 种预设材质。
纹理映射：

- 支持 BMP、PNG、JPG、JPEG、GIF、TIFF 等格式
- 使用 Windows Imaging Component (WIC) 加载图像
- 支持颜色混合和纹理替换两种映射模式

3.2.5 用户交互模块

模块功能：
本模块实现三维场景的交互式操作，包括图形选择、移动和摄像机控制。
鼠标拾取 **(Ray Picking)**：

- 将屏幕坐标转换为 NDC 坐标
- 通过逆投影矩阵和逆视图矩阵计算射线方向
- 对每个物体进行射线-包围球相交测试
- 选择距离摄像机最近的相交物体

图形移动：

- 左键拖动：在 XY 平面内移动选中物体
- 滚轮：调整选中物体的 Z 轴位置
- 双击：打开属性编辑对话框

摄像机控制：

- 右键/中键拖动：旋转摄像机视角（球坐标系）
- 滚轮（无选中物体时）：调整摄像机距离
- 摄像机始终朝向原点

键盘操作：

- Delete 键：删除选中的图形

4 实验算法及程序流程

4.1 三维几何体生成算法

4.1.1 球体生成算法

输入：半径 r，经度细分数 $slices$，纬度细分数 $stacks$
输出：顶点数组和索引数组
算法原理：
使用球坐标系参数化方法，将球面划分为 $slices \times stacks$ 个四边形网格。
球面上任意一点的坐标为：

$$x = r \cdot \sin\phi \cdot \cos\theta$$
$$y = r \cdot \cos\phi$$
$$z = r \cdot \sin\phi \cdot \sin\theta$$

其中 $\phi \in [0, \pi]$ 为纬度角，$\theta \in [0, 2\pi]$ 为经度角。
法线向量等于归一化的位置向量：$\vec{N} = \frac{\vec{P}}{|\vec{P}|}$
纹理坐标：$u = \frac{slice}{slices}$，$v = \frac{stack}{stacks}$

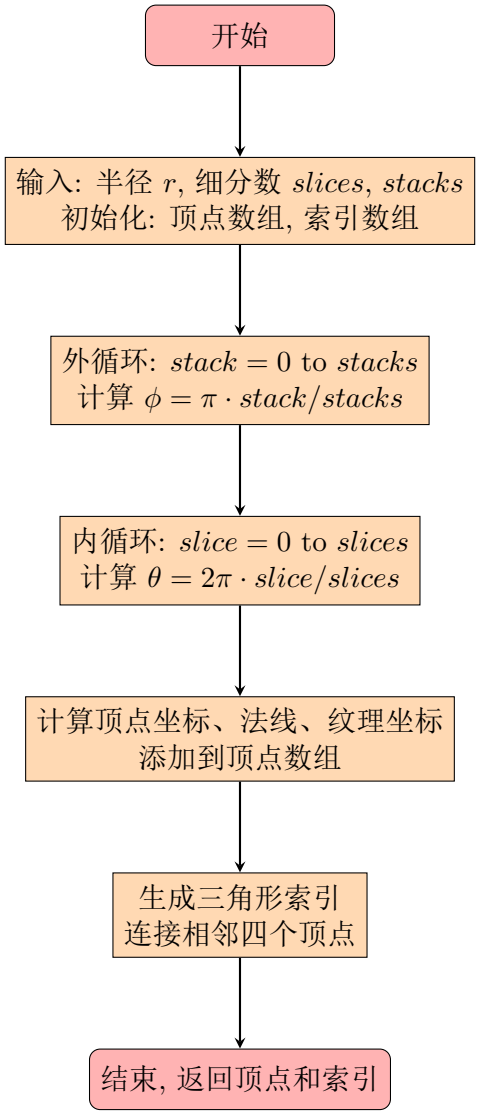


图 2: 球体生成算法流程图

4.1.2 圆柱体生成算法

输入：半径 r，高度 h，细分数 $slices$
输出：顶点数组和索引数组
算法原理：
圆柱体由三部分组成：顶面、底面和侧面。
顶面和底面：

- 圆心顶点：$(0, \pm h/2, 0)$，法线 $(0, \pm 1, 0)$
- 圆周顶点：$(r\cos\theta, \pm h/2, r\sin\theta)$

侧面：

- 顶点坐标：$(r\cos\theta, y, r\sin\theta)$，$y \in [-h/2, h/2]$
- 法线向量：$(\cos\theta, 0, \sin\theta)$（水平向外）

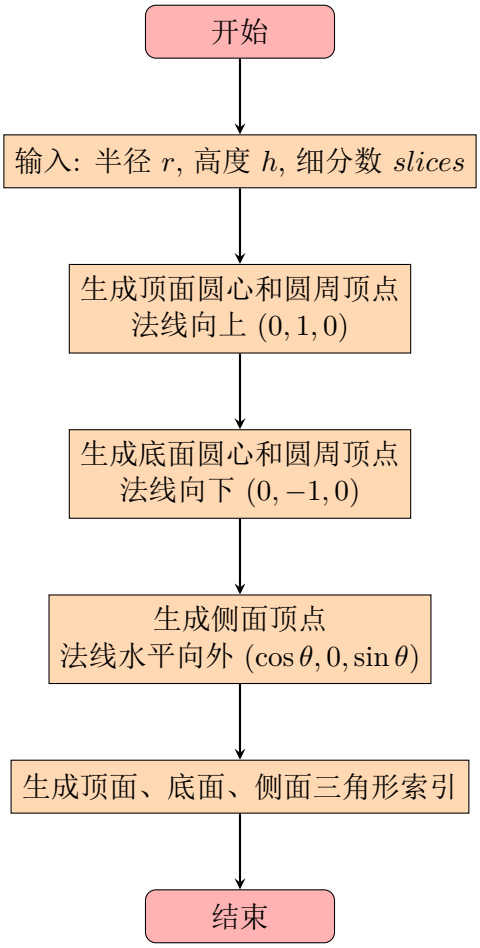


图 3: 圆柱体生成算法流程图

4.1.3 立方体生成算法

输入：边长 $size$
输出：顶点数组和索引数组
算法原理：
立方体有 6 个面，每个面 4 个顶点，共 24 个顶点（因为每个面需要独立的法线）。
六个面的法线：

- 前面：$(0, 0, -1)$，后面：$(0, 0, 1)$
- 左面：$(-1, 0, 0)$，右面：$(1, 0, 0)$

- 顶面：$(0, 1, 0)$，底面：$(0, -1, 0)$

每个面生成 2 个三角形，共 12 个三角形。

4.1.4 四面体生成算法

输入：边长 $size$
输出：顶点数组和索引数组
算法原理：
正四面体有 4 个顶点和 4 个三角形面。
顶点坐标：

- 顶点 v_0：$(0, h, 0)$，其中 $h = size \cdot \sqrt{2/3}$
- 底面三个顶点均匀分布在半径 $r = size/\sqrt{3}$ 的圆上

法线计算：每个面的法线通过两条边的叉积计算：

$$\vec{N} = \frac{(\vec{v_1} - \vec{v_0}) \times (\vec{v_2} - \vec{v_0})}{|(\vec{v_1} - \vec{v_0}) \times (\vec{v_2} - \vec{v_0})|}$$

4.2 三维图形变换算法

4.2.1 世界矩阵计算

输入：位置 (p_x, p_y, p_z)，旋转角度 $(\theta_x, \theta_y, \theta_z)$，缩放 (s_x, s_y, s_z)
输出：4×4 世界变换矩阵
算法流程：

1. 构建缩放矩阵 M_S
2. 构建 X 轴旋转矩阵 M_{Rx}
3. 构建 Y 轴旋转矩阵 M_{Ry}
4. 构建 Z 轴旋转矩阵 M_{Rz}
5. 构建平移矩阵 M_T
6. 计算复合矩阵：$M_{World} = M_S \cdot M_{Rx} \cdot M_{Ry} \cdot M_{Rz} \cdot M_T$

变换顺序说明：先缩放、再旋转、最后平移，确保旋转和缩放都是相对于物体自身中心进行的。

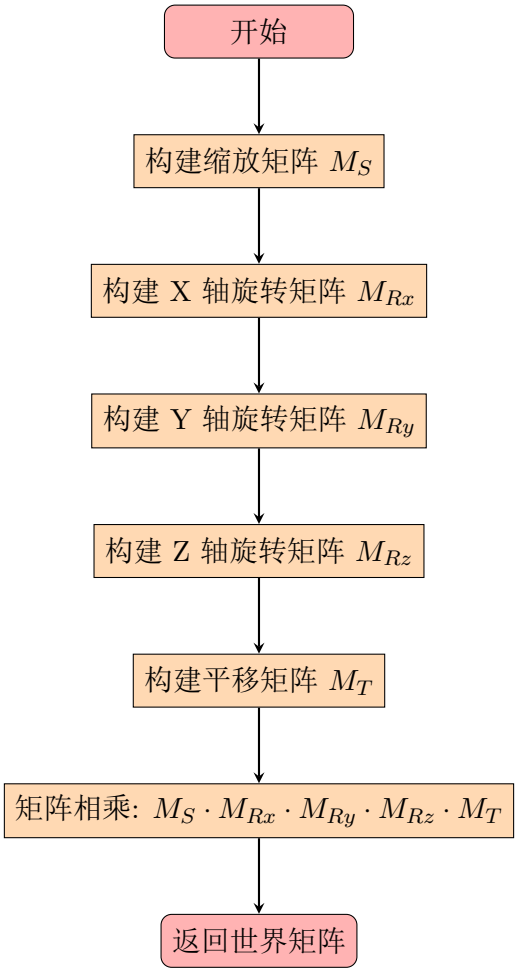


图 4: 世界矩阵计算流程图

4.3 摄像机视图矩阵算法

输入：摄像机偏航角 yaw，俯仰角 $pitch$，距离 $distance$
输出：视图矩阵和摄像机位置
算法原理：
使用球坐标系计算摄像机位置，摄像机始终朝向原点。
摄像机位置计算：

$$Eye_x = distance \cdot \cos(pitch) \cdot \cos(yaw)$$
$$Eye_y = distance \cdot \cos(pitch) \cdot \sin(yaw)$$
$$Eye_z = distance \cdot \sin(pitch)$$

视图矩阵：使用 LookAt 函数构建

$$M_{View} = LookAt(Eye, Target, Up)$$

其中 $Target = (0, 0, 0)$，$Up = (0, 0, 1)$。

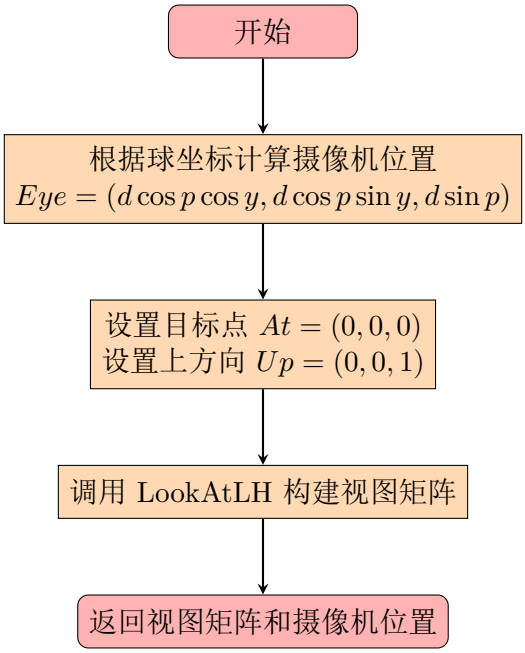


图 5: 摄像机视图矩阵计算流程图

4.4 射线拾取算法

输入：鼠标屏幕坐标 (x, y)
输出：被选中的图形对象（或空）
算法原理：
将屏幕坐标转换为三维射线，然后与场景中的物体进行相交测试。
详细步骤：

1. 屏幕坐标转 **NDC**：

$$ndcX = \frac{2x}{width} - 1, \quad ndcY = 1 - \frac{2y}{height}$$

2. 计算射线方向：

   - 构建近平面点：$(ndcX, ndcY, 1)$
   - 通过逆投影矩阵变换到视图空间

- 通过逆视图矩阵变换到世界空间
- 归一化得到射线方向

3. 射线-包围球相交测试：
   - 将射线变换到物体局部坐标系
   - 计算射线到原点的最近点
   - 判断最近点距离是否小于包围球半径
4. 选择最近物体：返回距离摄像机最近的相交物体

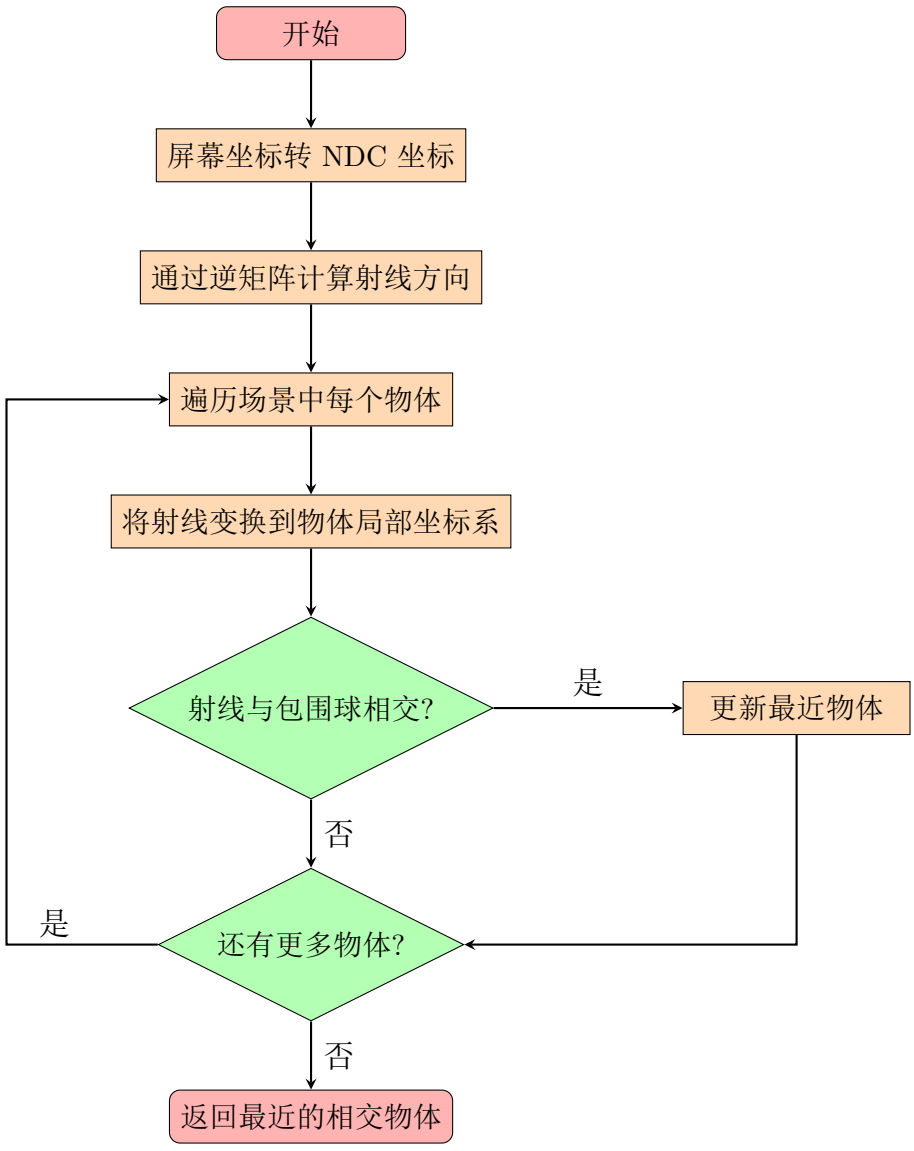


图 6: 射线拾取算法流程图

5 程序实现

5.1 算法设计思路

5.1.1 Direct3D 11 渲染管线设计

本程序采用 Direct3D 11 图形 API，实现了完整的渲染管线：
渲染管线流程：

1. 输入装配 **(IA)**：设置顶点缓冲区、索引缓冲区、输入布局
2. 顶点着色器 **(VS)**：进行世界、视图、投影变换
3. 光栅化 **(RS)**：三角形裁剪、背面剔除、插值
4. 像素着色器 **(PS)**：计算 Phong 光照、纹理采样
5. 输出合并 **(OM)**：深度测试、Alpha 混合

5.1.2 着色器设计思路

顶点着色器：

- 将顶点从模型空间变换到裁剪空间
- 将法线变换到世界空间（用于光照计算）
- 传递颜色和纹理坐标到像素着色器

像素着色器：

- 归一化插值后的法线向量
- 计算 Phong 光照模型的各个分量
- 根据纹理模式进行纹理采样和混合
- 输出最终颜色

5.1.3 材质系统设计

材质系统采用预设 + 自定义的混合模式：

- 提供 11 种常用材质预设，方便快速选择
- 支持自定义所有材质参数
- 材质参数通过常量缓冲区传递给着色器

5.2 核心代码实现

5.2.1 顶点和常量缓冲区结构

代码思路：定义 GPU 和 CPU 之间传递数据的结构体。

```
// 顶点结构 - 包含位置、法线、颜色、纹理坐标
struct Vertex {
    XMFLOAT3 position;  // 三维位置
    XMFLOAT3 normal;    // 法线向量
    XMFLOAT4 color;     // RGBA颜色
    XMFLOAT2 texCoord;  // 纹理坐标
};

// 常量缓冲区 - 传递变换矩阵和光照参数
struct ConstantBuffer {
    XMMATRIX world;        // 世界矩阵
    XMMATRIX view;         // 视图矩阵
    XMMATRIX projection;   // 投影矩阵
    XMFLOAT4 lightDir;     // 光源方向
    XMFLOAT4 lightColor;   // 光源颜色
    XMFLOAT4 cameraPos;    // 摄像机位置
    XMFLOAT4 matAmbient;   // 环境光系数
    XMFLOAT4 matDiffuse;   // 漫反射系数
    XMFLOAT4 matSpecular; // 镜面反射系数(w=光泽度)
    XMFLOAT4 matEmissive; // 自发光系数
    XMFLOAT4 textureInfo; // 纹理模式信息
    XMFLOAT4 materialExtra; // 透明度、反射率等
};
```

Listing 1: 顶点和常量缓冲区结构定义

5.2.2 世界矩阵计算

代码思路：组合缩放、旋转、平移矩阵得到世界变换矩阵。

```
// 计算物体的世界变换矩阵
// 变换顺序: 缩放 -> 旋转X -> 旋转Y -> 旋转Z -> 平移
XMMATRIX GetWorldMatrix() {
    // 构建各个变换矩阵
    XMMATRIX mScale = XMMatrixScaling(scale.x, scale.y, scale.z);
```

```
    XMMATRIX mRotX = XMMatrixRotationX(rotation.x);
    XMMATRIX mRotY = XMMatrixRotationY(rotation.y);
    XMMATRIX mRotZ = XMMatrixRotationZ(rotation.z);
    XMMATRIX mTrans = XMMatrixTranslation(
        position.x, position.y, position.z);

    // 按顺序相乘得到世界矩阵
    return mScale * mRotX * mRotY * mRotZ * mTrans;
}
```

Listing 2: 世界矩阵计算函数

5.2.3 球体生成实现

代码思路：使用球坐标系参数化方法，通过双重循环生成球面网格。

```
// 创建球体网格
// radius: 半径, slices: 经度细分, stacks: 纬度细分
void CreateSphere(float radius, int slices, int stacks,
    std::vector<Vertex>& vertices, std::vector<UINT>& indices) {
    vertices.clear();
    indices.clear();

    // 生成顶点
    for (int stack = 0; stack <= stacks; ++stack) {
        float phi = XM_PI * stack / stacks;  // 纬度角 [0, PI]
        for (int slice = 0; slice <= slices; ++slice) {
            float theta = 2.0f * XM_PI * slice / slices;  // 经度
                角

            Vertex v;
            // 球坐标转直角坐标
            v.position.x = radius * sinf(phi) * cosf(theta);
            v.position.y = radius * cosf(phi);
            v.position.z = radius * sinf(phi) * sinf(theta);

            // 法线 = 归一化的位置向量
            XMVECTOR pos = XMLoadFloat3(&v.position);
            XMVECTOR norm = XMVector3Normalize(pos);
```

```
            XMStoreFloat3(&v.normal, norm);

            v.color = XMFLOAT4(0.7f, 0.7f, 0.7f, 1.0f);
            v.texCoord = XMFLOAT2((float)slice/slices,
                                  (float)stack/stacks);
            vertices.push_back(v);
        }
    }

    // 生成索引 - 每个四边形分成两个三角形
    for (int stack = 0; stack < stacks; ++stack) {
        for (int slice = 0; slice < slices; ++slice) {
            int first = stack * (slices + 1) + slice;
            int second = first + slices + 1;
            // 第一个三角形
            indices.push_back(first);
            indices.push_back(first + 1);
            indices.push_back(second);
            // 第二个三角形
            indices.push_back(second);
            indices.push_back(first + 1);
            indices.push_back(second + 1);
        }
    }
}
```

Listing 3: 球体生成函数

5.2.4 圆柱体生成实现

代码思路：分别生成顶面、底面和侧面，注意各部分的法线方向。

```
// 创建圆柱体网格
void CreateCylinder(float radius, float height, int slices,
    std::vector<Vertex>& vertices, std::vector<UINT>& indices) {
    vertices.clear();
    indices.clear();
    float halfHeight = height * 0.5f;
    XMFLOAT4 defaultColor(0.7f, 0.7f, 0.7f, 1.0f);
```

```
8
9       // 顶面圆心
10      Vertex topCenter;
11      topCenter.position = XMFLOAT3(0, halfHeight, 0);
12      topCenter.normal = XMFLOAT3(0, 1, 0);  // 法线向上
13      topCenter.color = defaultColor;
14      vertices.push_back(topCenter);
15
16      // 底面圆心
17      Vertex bottomCenter;
18      bottomCenter.position = XMFLOAT3(0, -halfHeight, 0);
19      bottomCenter.normal = XMFLOAT3(0, -1, 0);  // 法线向下
20      bottomCenter.color = defaultColor;
21      vertices.push_back(bottomCenter);
22
23      // 顶部圆周顶点(用于侧面)
24      for (int i = 0; i <= slices; ++i) {
25          float theta = 2.0f * XM_PI * i / slices;
26          Vertex v;
27          v.position.x = radius * cosf(theta);
28          v.position.y = halfHeight;
29          v.position.z = radius * sinf(theta);
30          // 侧面法线水平向外
31          v.normal = XMFLOAT3(cosf(theta), 0, sinf(theta));
32          v.color = defaultColor;
33          v.texCoord = XMFLOAT2((float)i / slices, 0.0f);
34          vertices.push_back(v);
35      }
36
37      // 底部圆周顶点(用于侧面)
38      for (int i = 0; i <= slices; ++i) {
39          float theta = 2.0f * XM_PI * i / slices;
40          Vertex v;
41          v.position.x = radius * cosf(theta);
42          v.position.y = -halfHeight;
43          v.position.z = radius * sinf(theta);
44          v.normal = XMFLOAT3(cosf(theta), 0, sinf(theta));
45          v.color = defaultColor;
```

```
        v.texCoord = XMFLOAT2((float)i / slices, 1.0f);
        vertices.push_back(v);
    }

    // 生成顶面、底面、侧面索引...
}
```

Listing 4: 圆柱体生成函数

5.2.5 四面体生成实现

代码思路：计算正四面体的四个顶点坐标，每个面独立计算法线。

```
// 创建正四面体网格
void CreateTetrahedron(float size,
    std::vector<Vertex>& vertices, std::vector<UINT>& indices) {
    vertices.clear();
    indices.clear();
    XMFLOAT4 color = XMFLOAT4(0.7f, 0.7f, 0.7f, 1.0f);

    // 计算四面体几何参数
    float h = size * sqrtf(2.0f / 3.0f);  // 高度
    float r = size / sqrtf(3.0f);          // 底面外接圆半径

    // 四个顶点坐标
    XMFLOAT3 v0(0, h, 0);                              // 顶点
    XMFLOAT3 v1(-size * 0.5f, 0, r);                   // 底面顶点1
    XMFLOAT3 v2(size * 0.5f, 0, r);                    // 底面顶点2
    XMFLOAT3 v3(0, 0, -size / sqrtf(3.0f));            // 底面顶点3

    // 计算第一个面的法线 (v0, v1, v2)
    XMVECTOR n1 = XMVector3Normalize(XMVector3Cross(
        XMVectorSubtract(XMLoadFloat3(&v1), XMLoadFloat3(&v0)),
        XMVectorSubtract(XMLoadFloat3(&v2), XMLoadFloat3(&v0))));
    XMFLOAT3 normal1;
    XMStoreFloat3(&normal1, n1);

    // 添加第一个面的三个顶点
    vertices.push_back({ v0, normal1, color, XMFLOAT2(0.5f, 0.0f)
```

```
        });
    vertices.push_back({ v1, normal1, color, XMFLOAT2(0.0f, 1.0f)
        });
    vertices.push_back({ v2, normal1, color, XMFLOAT2(1.0f, 1.0f)
        });

    // 类似地计算其他三个面...
    // 每个面独立的法线确保正确的光照效果

    // 生成索引
    for (UINT i = 0; i < 12; ++i) {
        indices.push_back(i);
    }
}
```

Listing 5: 四面体生成函数

5.2.6 顶点着色器实现

代码思路：将顶点从模型空间变换到裁剪空间，同时传递光照计算所需的数据。

```
// 常量缓冲区
cbuffer ConstantBuffer : register(b0) {
    matrix World; matrix View; matrix Projection;
    float4 LightDir; float4 LightColor; float4 CameraPos;
    float4 MatAmbient; float4 MatDiffuse;
    float4 MatSpecular; float4 MatEmissive;
    float4 TextureInfo; float4 MaterialExtra;
}

// 输入输出结构
struct VS_INPUT {
    float3 Pos : POSITION;
    float3 Normal : NORMAL;
    float4 Color : COLOR;
    float2 TexCoord : TEXCOORD;
};
struct PS_INPUT {
    float4 Pos : SV_POSITION;
```

```
    float3 Normal : NORMAL;
    float4 Color : COLOR;
    float3 WorldPos : TEXCOORD0;
    float2 TexCoord : TEXCOORD1;
};

PS_INPUT main(VS_INPUT input) {
    PS_INPUT output;
    // 计算世界坐标
    float4 worldPos = mul(float4(input.Pos, 1.0f), World);
    output.WorldPos = worldPos.xyz;
    // 变换到裁剪空间
    output.Pos = mul(mul(worldPos, View), Projection);
    // 变换法线到世界空间
    output.Normal = mul(input.Normal, (float3x3)World);
    output.Color = input.Color;
    output.TexCoord = input.TexCoord;
    return output;
}
```

Listing 6: HLSL 顶点着色器

5.2.7 像素着色器实现

代码思路：实现 Phong 光照模型，支持纹理采样和多种混合模式。

```
Texture2D texDiffuse : register(t0);
SamplerState samLinear : register(s0);

float4 main(PS_INPUT input) : SV_Target {
    // 归一化法线
    float3 normal = normalize(input.Normal);
    // 计算光线方向(指向光源)
    float3 lightDir = normalize(-LightDir.xyz);

    // 基础颜色
    float3 baseColor = input.Color.rgb;
    float alpha = input.Color.a;

```

```
    // 纹理采样
    int texMode = (int)TextureInfo.x;
    if (texMode > 0) {
        float4 texColor = texDiffuse.Sample(samLinear, input.
           TexCoord);
        if (texMode == 1) {  // 颜色混合模式
            baseColor = texColor.rgb * MatDiffuse.rgb;
            alpha = texColor.a * input.Color.a;
        } else if (texMode == 2) {  // 纹理替换模式
            baseColor = texColor.rgb;
            alpha = texColor.a;
        }
    }

    // Phong光照计算
    // 环境光
    float3 ambient = MatAmbient.rgb * LightColor.rgb;
    // 漫反射
    float NdotL = max(dot(normal, lightDir), 0.0f);
    float3 diffuse = MatDiffuse.rgb * NdotL * LightColor.rgb;
    // 镜面反射
    float3 viewDir = normalize(CameraPos.xyz - input.WorldPos);
    float3 reflectDir = reflect(-lightDir, normal);
    float spec = pow(max(dot(viewDir, reflectDir), 0.0f),
       MatSpecular.w);
    float3 specular = MatSpecular.rgb * spec * LightColor.rgb;
    // 自发光
    float3 emissive = MatEmissive.rgb;

    // 最终颜色
    float3 finalColor = (ambient + diffuse + specular) *
       baseColor + emissive;
    return float4(saturate(finalColor), alpha);
}
```

Listing 7: HLSL 像素着色器

5.2.8 摄像机视图矩阵实现

代码思路：使用球坐标系计算摄像机位置，始终朝向原点。

```
// 计算摄像机视图矩阵
// 使用球坐标系: yaw(偏航角), pitch(俯仰角), distance(距离)
void GetCameraViewMatrix(XMMATRIX& outView, XMVECTOR& outEye) {
    // 球坐标转直角坐标计算摄像机位置
    outEye = XMVectorSet(
        g_cameraDistance * cosf(g_cameraPitch) * cosf(g_cameraYaw
            ),
        g_cameraDistance * cosf(g_cameraPitch) * sinf(g_cameraYaw
            ),
        g_cameraDistance * sinf(g_cameraPitch),
        0.0f);

    // 目标点为原点
    XMVECTOR At = XMVectorSet(0.0f, 0.0f, 0.0f, 0.0f);
    // 上方向为Z轴正方向
    XMVECTOR Up = XMVectorSet(0.0f, 0.0f, 1.0f, 0.0f);

    // 构建左手坐标系的LookAt矩阵
    outView = XMMatrixLookAtLH(outEye, At, Up);
}
```

Listing 8: 摄像机视图矩阵计算

5.2.9 射线拾取实现

代码思路：将屏幕坐标转换为三维射线，与物体包围球进行相交测试。

```
// 根据鼠标位置拾取物体
Shape3D* PickShape(int x, int y) {
    // 屏幕坐标转NDC
    float ndcX = (2.0f * x / WINDOW_WIDTH) - 1.0f;
    float ndcY = 1.0f - (2.0f * y / WINDOW_HEIGHT);

    // 构建投影矩阵和逆矩阵
    XMMATRIX projection = XMMatrixPerspectiveFovLH(
        XM_PIDIV4, (float)WINDOW_WIDTH / WINDOW_HEIGHT, 0.01f,
            100.0f);
```

```
    XMMATRIX invProj = XMMatrixInverse(nullptr, projection);

    // 计算射线方向
    XMVECTOR rayDir = XMVectorSet(ndcX, ndcY, 1.0f, 0.0f);
    rayDir = XMVector3TransformCoord(rayDir, invProj);

    XMVECTOR Eye; XMMATRIX view;
    GetCameraViewMatrix(view, Eye);
    XMMATRIX invView = XMMatrixInverse(nullptr, view);

    XMVECTOR rayOrigin = Eye;
    rayDir = XMVector3TransformNormal(rayDir, invView);
    rayDir = XMVector3Normalize(rayDir);

    // 遍历所有物体进行相交测试
    Shape3D* closest = nullptr;
    float minDist = FLT_MAX;
    for (auto shape : g_shapes) {
        // 射线-包围球相交测试...
        // 选择最近的相交物体
    }
    return closest;
}
```

Listing 9: 射线拾取函数

5.2.10 材质预设实现

代码思路：定义多种常用材质的光照参数，方便用户快速选择。

```
// 应用预设材质样式
void ApplyPresetStyle(PresetStyle style) {
    presetStyle = style;
    switch (style) {
    case STYLE_GOLD:  // 黄金材质
        color = XMFLOAT4(1.0f, 0.84f, 0.0f, 1.0f);
        material.ambient = XMFLOAT3(0.24725f, 0.1995f, 0.0745f);
        material.diffuse = XMFLOAT3(0.75164f, 0.60648f, 0.22648f)
            ;
```

```
        material.specular = XMFLOAT3(0.628281f, 0.555802f,
            0.366065f);
        material.emissive = XMFLOAT3(0.0f, 0.0f, 0.0f);
        material.shininess = 51.2f;
        material.transparency = 1.0f;
        material.reflectivity = 0.7f;
        break;

    case STYLE_GLASS:  // 玻璃材质
        color = XMFLOAT4(0.9f, 0.95f, 1.0f, 0.3f);
        material.ambient = XMFLOAT3(0.1f, 0.1f, 0.1f);
        material.diffuse = XMFLOAT3(0.6f, 0.7f, 0.8f);
        material.specular = XMFLOAT3(1.0f, 1.0f, 1.0f);
        material.shininess = 96.0f;
        material.transparency = 0.3f;  // 半透明
        material.reflectivity = 0.8f;
        break;

    case STYLE_CHROME:  // 铬材质
        color = XMFLOAT4(0.9f, 0.9f, 0.9f, 1.0f);
        material.ambient = XMFLOAT3(0.25f, 0.25f, 0.25f);
        material.diffuse = XMFLOAT3(0.4f, 0.4f, 0.4f);
        material.specular = XMFLOAT3(0.774597f, 0.774597f,
            0.774597f);
        material.shininess = 76.8f;
        material.reflectivity = 0.9f;
        break;
    // ... 其他材质预设
    }
}
```

Listing 10: 材质预设应用函数

5.2.11 纹理加载实现

代码思路：使用 Windows Imaging Component (WIC) 加载各种格式的图像文件。

```
// 使用WIC加载图片纹理
bool LoadTextureFromFile(const wchar_t* filename,
```

```
    ID3D11ShaderResourceView** textureView) {
    if (!g_pd3dDevice || !filename || !textureView) return false;
    *textureView = nullptr;

    // 初始化COM
    HRESULT hr = CoInitializeEx(nullptr, COINIT_MULTITHREADED);

    // 创建WIC工厂
    IWICImagingFactory* pFactory = nullptr;
    hr = CoCreateInstance(CLSID_WICImagingFactory, nullptr,
        CLSCTX_INPROC_SERVER, __uuidof(IWICImagingFactory),
        (void**)&pFactory);

    // 创建解码器并加载图像
    IWICBitmapDecoder* pDecoder = nullptr;
    hr = pFactory->CreateDecoderFromFilename(filename, nullptr,
        GENERIC_READ, WICDecodeMetadataCacheOnLoad, &pDecoder);

    // 获取第一帧并转换为32位RGBA格式
    IWICBitmapFrameDecode* pFrame = nullptr;
    pDecoder->GetFrame(0, &pFrame);

    IWICFormatConverter* pConverter = nullptr;
    pFactory->CreateFormatConverter(&pConverter);
    pConverter->Initialize(pFrame, GUID_WICPixelFormat32bppRGBA,
        WICBitmapDitherTypeNone, nullptr, 0.0,
        WICBitmapPaletteTypeCustom);

    // 读取像素数据
    UINT width, height;
    pFrame->GetSize(&width, &height);
    std::vector<BYTE> imageData(width * height * 4);
    pConverter->CopyPixels(nullptr, width * 4,
        width * height * 4, imageData.data());

    // 创建D3D11纹理
    D3D11_TEXTURE2D_DESC texDesc = {};
    texDesc.Width = width;
```

```
    texDesc.Height = height;
    texDesc.MipLevels = 1;
    texDesc.ArraySize = 1;
    texDesc.Format = DXGI_FORMAT_R8G8B8A8_UNORM;
    texDesc.SampleDesc.Count = 1;
    texDesc.Usage = D3D11_USAGE_DEFAULT;
    texDesc.BindFlags = D3D11_BIND_SHADER_RESOURCE;

    // 创建纹理和着色器资源视图...
    return SUCCEEDED(hr);
}
```

Listing 11: 纹理加载函数

5.2.12 渲染循环实现

代码思路：每帧清除缓冲区，设置渲染状态，遍历所有物体进行绘制。

```
void Render() {
    if (!g_pImmediateContext) return;

    // 清除渲染目标和深度缓冲
    float clearColor[4] = { 0.75f, 0.75f, 0.78f, 1.0f };
    g_pImmediateContext->ClearRenderTargetView(
        g_pRenderTargetView, clearColor);
    g_pImmediateContext->ClearDepthStencilView(
        g_pDepthStencilView, D3D11_CLEAR_DEPTH, 1.0f, 0);

    // 获取摄像机矩阵
    XMVECTOR Eye; XMMATRIX view;
    GetCameraViewMatrix(view, Eye);
    XMMATRIX projection = XMMatrixPerspectiveFovLH(
        XM_PIDIV4, (float)WINDOW_WIDTH / WINDOW_HEIGHT,
        0.01f, 100000.0f);

    // 设置渲染状态
    g_pImmediateContext->IASetInputLayout(g_pVertexLayout);
    g_pImmediateContext->VSSetShader(g_pVertexShader, nullptr, 0)
        ;
```

```
    g_pImmediateContext->PSSetShader(g_pPixelShader, nullptr, 0);
    g_pImmediateContext->RSSetState(g_pRasterizerState);
    g_pImmediateContext->OMSetDepthStencilState(
        g_pDepthStencilState, 1);
    g_pImmediateContext->OMSetBlendState(g_pBlendState, nullptr,
        0xffffffff);

    // 遍历所有物体进行绘制
    for (auto shape : g_shapes) {
        // 设置常量缓冲区(世界矩阵、材质参数等)
        ConstantBuffer cb;
        cb.world = XMMatrixTranspose(shape->GetWorldMatrix());
        cb.view = XMMatrixTranspose(view);
        cb.projection = XMMatrixTranspose(projection);
        // ... 设置光照和材质参数

        g_pImmediateContext->UpdateSubresource(
            g_pConstantBuffer, 0, nullptr, &cb, 0, 0);

        // 绑定顶点和索引缓冲区
        UINT stride = sizeof(Vertex), offset = 0;
        g_pImmediateContext->IASetVertexBuffers(
            0, 1, &shape->vertexBuffer, &stride, &offset);
        g_pImmediateContext->IASetIndexBuffer(
            shape->indexBuffer, DXGI_FORMAT_R32_UINT, 0);

        // 绘制
        g_pImmediateContext->DrawIndexed(shape->indexCount, 0, 0)
            ;

        // 选中物体绘制线框高亮
        if (shape->selected) {
            g_pImmediateContext->RSSetState(g_pWireframeState);
            g_pImmediateContext->DrawIndexed(shape->indexCount,
                0, 0);
            g_pImmediateContext->RSSetState(g_pRasterizerState);
        }
    }
```

```
55
56      // 绘制坐标轴
57      if (g_showAxes) { /* 绘制XYZ坐标轴 */ }
58
59      // 呈现
60      g_pSwapChain->Present(0, 0);
61  }
```

Listing 12: 主渲染函数

6 实验总结

通过本次实验，深入理解了计算机图形学中三维图形生成与变换的原理和实现方法：

1. 三维几何体生成：掌握了球体、圆柱体、立方体、四面体等基本几何体的参数化生成方法。理解了顶点坐标、法线向量、纹理坐标的计算原理，特别是法线向量对光照效果的重要影响。

2. 三维变换矩阵：深入理解了齐次坐标和 4×4 变换矩阵的概念。掌握了平移、旋转、缩放矩阵的构建方法，以及复合变换的矩阵乘法顺序（缩放 → 旋转 → 平移）。

3. **Direct3D 11** 渲染管线：学习了现代图形 API 的渲染管线架构，包括输入装配、顶点着色器、光栅化、像素着色器、输出合并等阶段。理解了 GPU 编程的基本概念。

4. **Phong** 光照模型：实现了完整的 Phong 光照模型，包括环境光、漫反射、镜面反射和自发光四个分量。理解了各个分量的物理意义和计算方法。

5. 材质与纹理：学习了材质属性（环境光系数、漫反射系数、镜面反射系数、光泽度等）对渲染效果的影响。掌握了使用 WIC 加载图像文件并创建纹理的方法。

6. 交互技术：实现了射线拾取算法，理解了屏幕坐标到三维射线的转换过程。掌握了球坐标系摄像机控制的实现方法。

7. **HLSL** 着色器编程：学习了 HLSL 着色器语言的基本语法，理解了顶点着色器和像素着色器的职责分工，以及常量缓冲区、纹理采样器的使用方法。

本实验综合运用了计算机图形学的多个核心概念，从几何建模到光照渲染，从变换矩阵到交互操作，构建了一个功能完整的三维图形编辑器。通过实践加深了对三维图形学理论的理解，提高了使用现代图形 API 进行图形编程的能力。